COS 350 Lecture 14

unix: kill

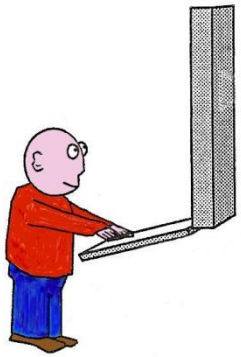
signals

signal handlers

sleep

alarm

pause



kill

- watch
- emacs
- vim
- bash (other)
- bash (self)

NAME

kill - send a signal to a process

SYNOPSIS

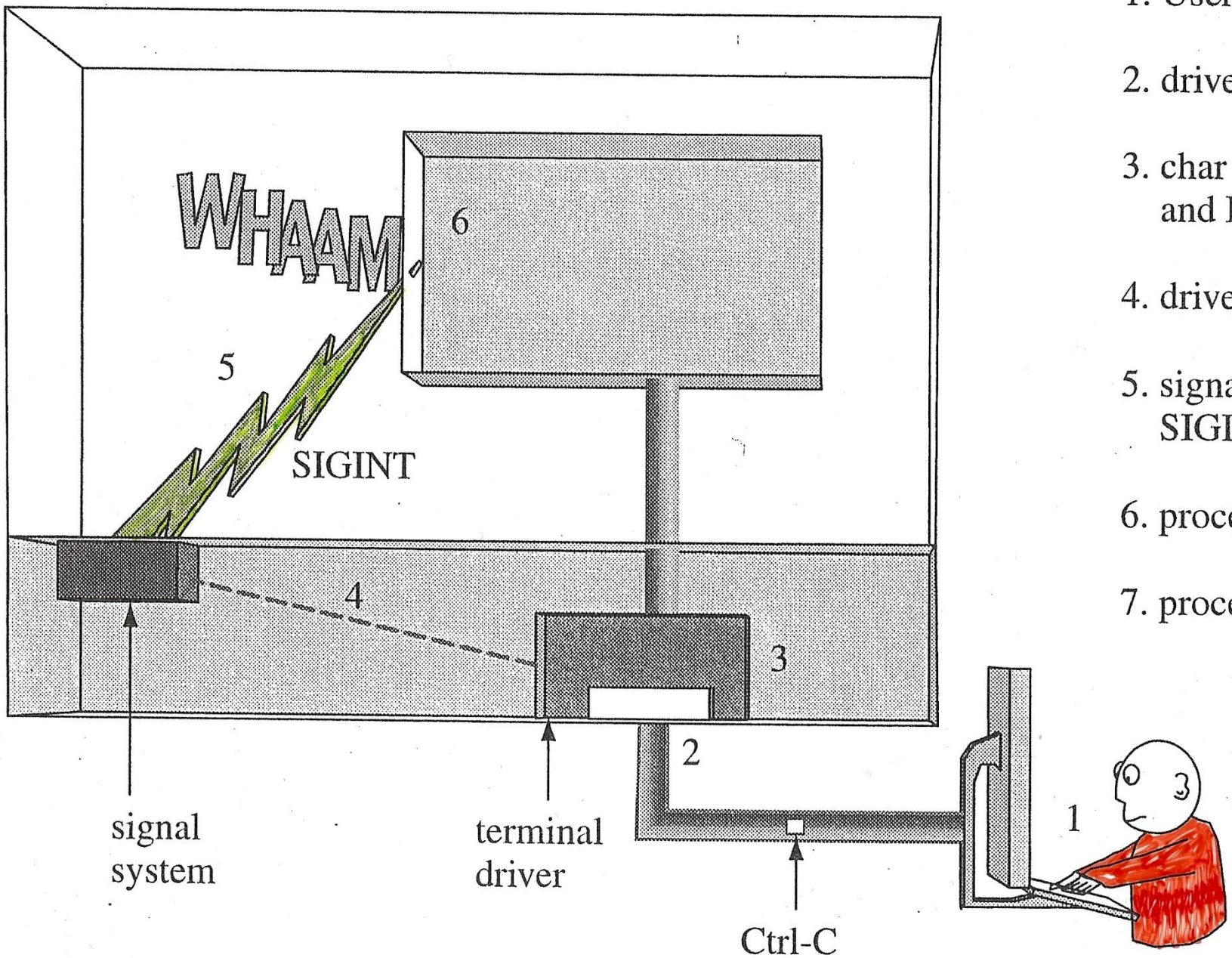
kill [-signal | -s signal] pid …
kill -l [signal]

DESCRIPTION

The default signal for kill is TERM. Use -l or -L to list available signals. Signals may be specified in three ways: -9 -SIGKILL -KILL.

NOTES

Your shell (command line interpreter) may have a built-in kill command. You may need to run the command described here as /bin/kill to solve the conflict.



1. User presses Ctrl-C
2. driver receives char
3. char matches VINTR and ISIG is on
4. driver calls signal system
5. signal system sends SIGINT to process
6. process receives SIGINT
7. process dies.

/usr/include/signal.h

```
#define SIGHUP      1   /* Hangup */
#define SIGINT      2   /* Interrupt */                  used by Ctrl-C
#define SIGQUIT     3   /* Quit */                       used by Ctrl-\
#define SIGILL      4   /* Illegal instruction */
#define SIGTRAP     5   /* Trace trap */
#define SIGABRT     6   /* Abort */
#define SIGBUS      7   /* BUS error */
#define SIGFPE      8   /* Floating-point exception */
#define SIGKILL     9   /* Kill, unblockable */
#define SIGUSR1     10  /* User-defined signal 1 */
#define SIGSEGV     11  /* Segmentation violation */
#define SIGUSR2     12  /* User-defined signal 2 */
#define SIGPIPE     13  /* Broken pipe */
#define SIGALRM     14  /* Alarm clock */
#define SIGTERM     15  /* Termination */                used by kill
#define SIGSTKFLT   16  /* Stack fault */
#define SIGCHLD     17  /* Child status has changed */
#define SIGCONT     18  /* Continue */
#define SIGSTOP     19  /* Stop, unblockable */
#define SIGTSTP     20  /* Keyboard stop */              used by Ctrl-Z
. . .
```

NAME

signal - ANSI C signal handling

SYNOPSIS

```
#include <signal.h>

typedef void (*sighandler_t)(int);

sighandler_t signal(int signum, sighandler_t handler);
```

DESCRIPTION

The behavior of signal() varies across Unix versions, and has also varied historically across different versions of Linux.

signal() sets the disposition of the signal signum to handler, which is either:

- SIG_IGN – ignore this signal
- SIG_DFL – reset to default handler
- or the address of a programmer-defined function (a "signal handler").

sigdemo1.c

```
// sigdemo1.c - shows how a signal handler works.
#include <stdio.h>
#include <signal.h>

/* signal handler function */
void f(int signum)
{
   printf("OUCH! (signal=%d)\n", signum);
}

int main()
{
   int i;

   signal( SIGINT, f );       /* install the handler      */

   for(i=0; i<60; i++ ){      /* do something for a while */
      printf("hello\n");
      sleep(1);
   }
}
```

Arrival of SIGINT diverts control flow to the signal handler. Return from the handler resumes previous control flow.

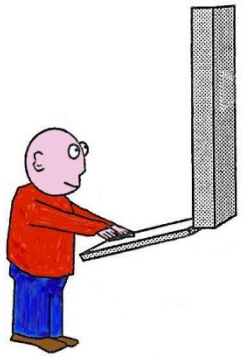
normal flow of control

```
main()
{
  signal(SIGINT, f);

  for(i=0; i<5; i++){
    printf("hello\n");
    sleep(1);
  }
}
```

flow to signal handler and back

```
f()
{
  printf("OUCH!");
}
```

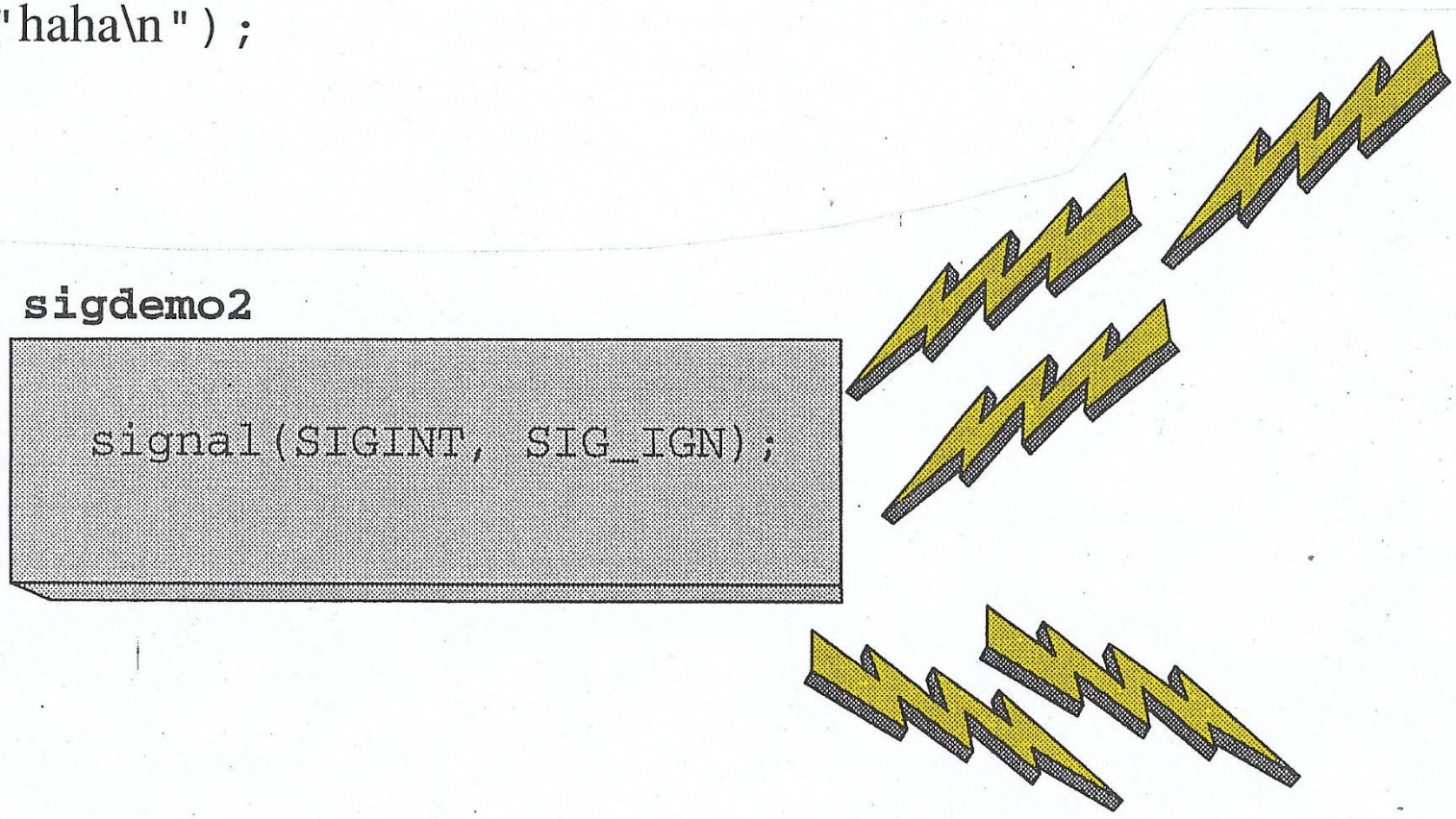


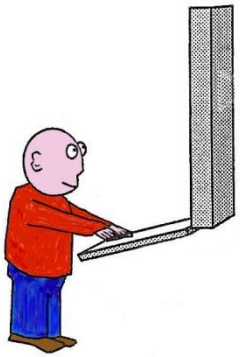
sigdemo1

```c
// sigdemo2.c - shows how to ignore a signal
#include <stdio.h>
#include <signal.h>

main()
{
  signal( SIGINT, SIG_IGN );

  printf("you can't stop me!\n");
  while( 1 )
    {
      sleep(1);
      printf("haha\n");
    }
}
```





segfault
password3
windowsize

Case study: sleep

```
unix> sleep 10
```

NAME

sleep - delay for a specified amount of time

SYNOPSIS

sleep NUMBER[SUFFIX]

DESCRIPTION

Pause for NUMBER seconds. SUFFIX may be 's' for seconds (the default), 'm' for minutes, 'h' for hours or 'd' for days.

Unlike most implementations that require NUMBER be an integer, here NUMBER may be an arbitrary floating point number.

NAME

sleep - sleep for the specified number of seconds

SYNOPSIS

```
#include <unistd.h>

unsigned int sleep(unsigned int seconds);
```

DESCRIPTION

sleep() makes the calling thread sleep until seconds seconds have elapsed or a signal arrives which is not ignored.

RETURN VALUE

Zero if the requested time has elapsed, or the number of seconds left to sleep, if the call was interrupted by a signal handler.

SEE ALSO

usleep(3), nanosleep(2), alarm(2)

NAME

alarm - set an alarm clock for delivery of a signal

SYNOPSIS

```
#include <unistd.h>

unsigned int alarm(unsigned int seconds);
```

DESCRIPTION

alarm() arranges for a SIGALRM signal to be delivered to the calling process in seconds seconds.

If seconds is zero, any pending alarm is canceled.

In any event any previously set alarm() is canceled.

RETURN VALUE

alarm() returns the number of seconds remaining until any previously scheduled alarm was due to be delivered, or zero if there was no previously scheduled alarm.

NAME

pause - wait for signal

SYNOPSIS

```
#include <unistd.h>

int pause(void);
```

DESCRIPTION

pause() causes the calling process (or thread) to sleep until a signal is delivered that either terminates the process or causes the invocation of a signal-catching function.

RETURN VALUE

pause() returns only when a signal was caught and the signal-catching function returned. In this case pause() returns -1, and errno is set to EINTR.

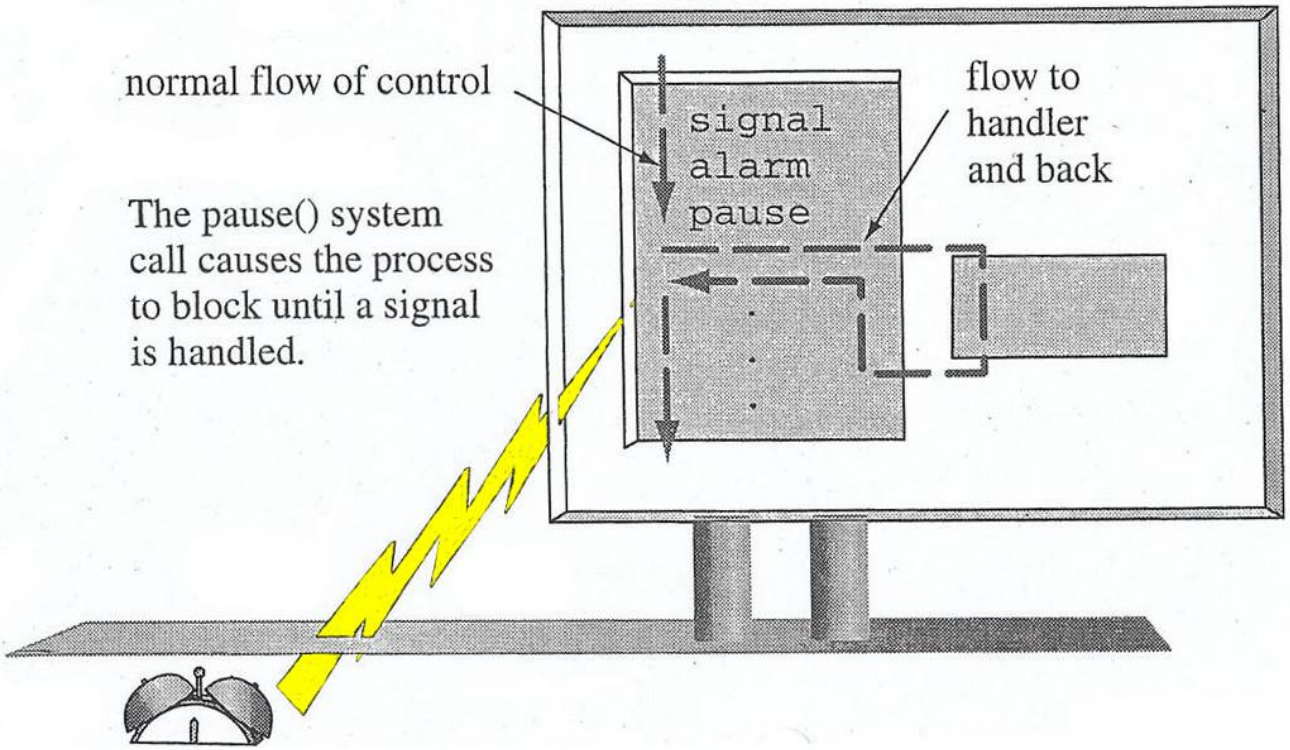
ERRORS

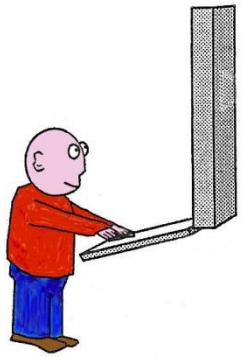
EINTR a signal was caught and the signal-catching function returned.

```c
// sleep1.c  sets handler, sets alarm, pauses, then returns
#include        <stdio.h>
#include        <signal.h>

void wakeup(int signum)
{
  printf("Alarm received from kernel\n");
}

main()
{
  printf("about to sleep for 4 seconds\n");
  signal(SIGALRM, wakeup);                    /* catch it     */
  alarm(4);                                   /* set clock    */
  pause();                                    /* freeze here  */
  printf("Morning so soon?\n");               /* back to work */
}
```





sleep1
password4.c
password4